AN2303
Application note

FR30xx SDK 目录结构与开发指南

简介

本应用手册具体介绍了在 FR30xx 系列芯片中，使用 SDK 时的注意事项与使用方法。

旨在减少研发工程师的开发时间，提高效率，缩短项目周期。

Version 1.0

目录

1. 获取 SDK 1
2. SDK 目录结构 2
2.1. 目录结构概述 2
2.2. components 文件结构 3
2.3. exammples 文件结构 4
3. 存储结构 5
3.1. 地址空间概述 5
3.2. FLASH 空间概述 6
4. 烧录 9
4.1. 串口烧录 9
4.2. 基于 Keil 烧录 12
4.3. 基于 J-Flash 烧录 15
5. System API 17
5.1. System API 概述 17
5.2. 外设时钟源获取 18
5.3. 微秒级别定时器 18
5.4. DMA 请求编号配置 18
5.5. 系统睡眠 21
6. 变更历史 22
7. 联系信息 23

Copyright© 2023 上海富芮坤微电子有限公司

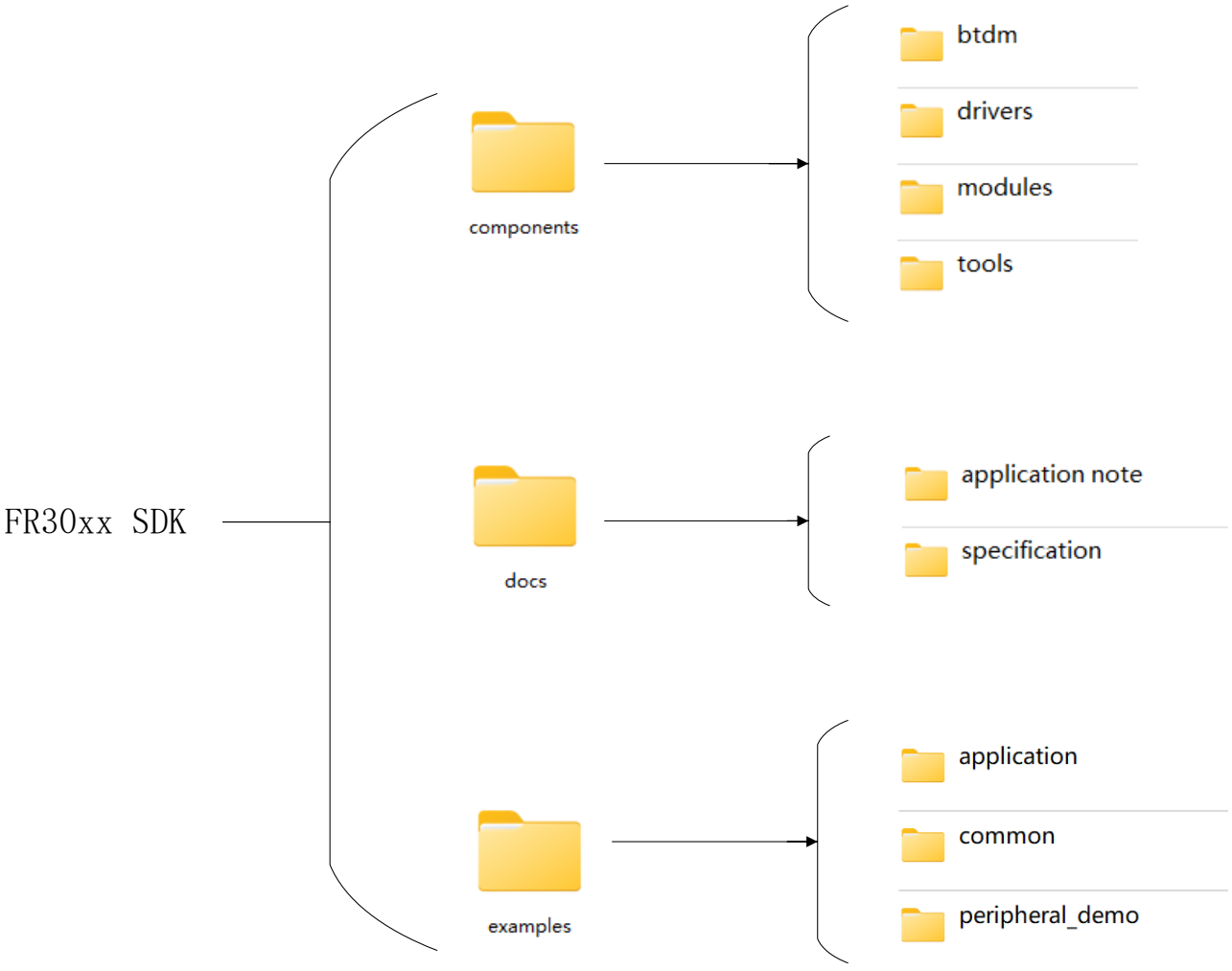
1. 获取 SDK

联系 FAE 获取

2. SDK 目录结构

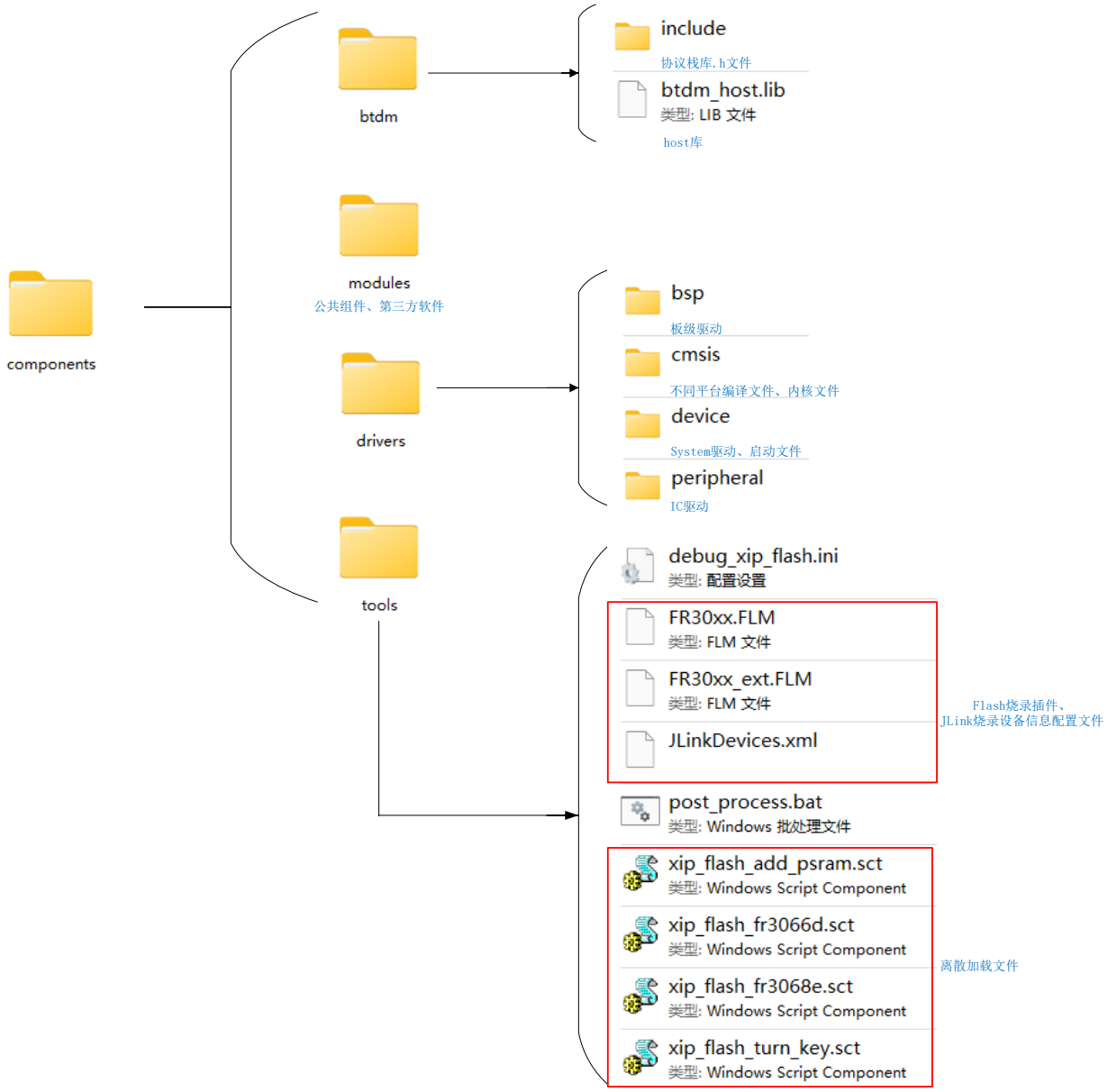
2.1. 目录结构概述

FR30xx SDK 的文件结构如下图所示。SDK 包含了MCU 外设驱动，应用层的例程代码，都是以源码的形式提供。蓝牙 host部分以库的形式提供。同时提供了完整的application note和specification手册、公共组件、第三方软件、工具文件。



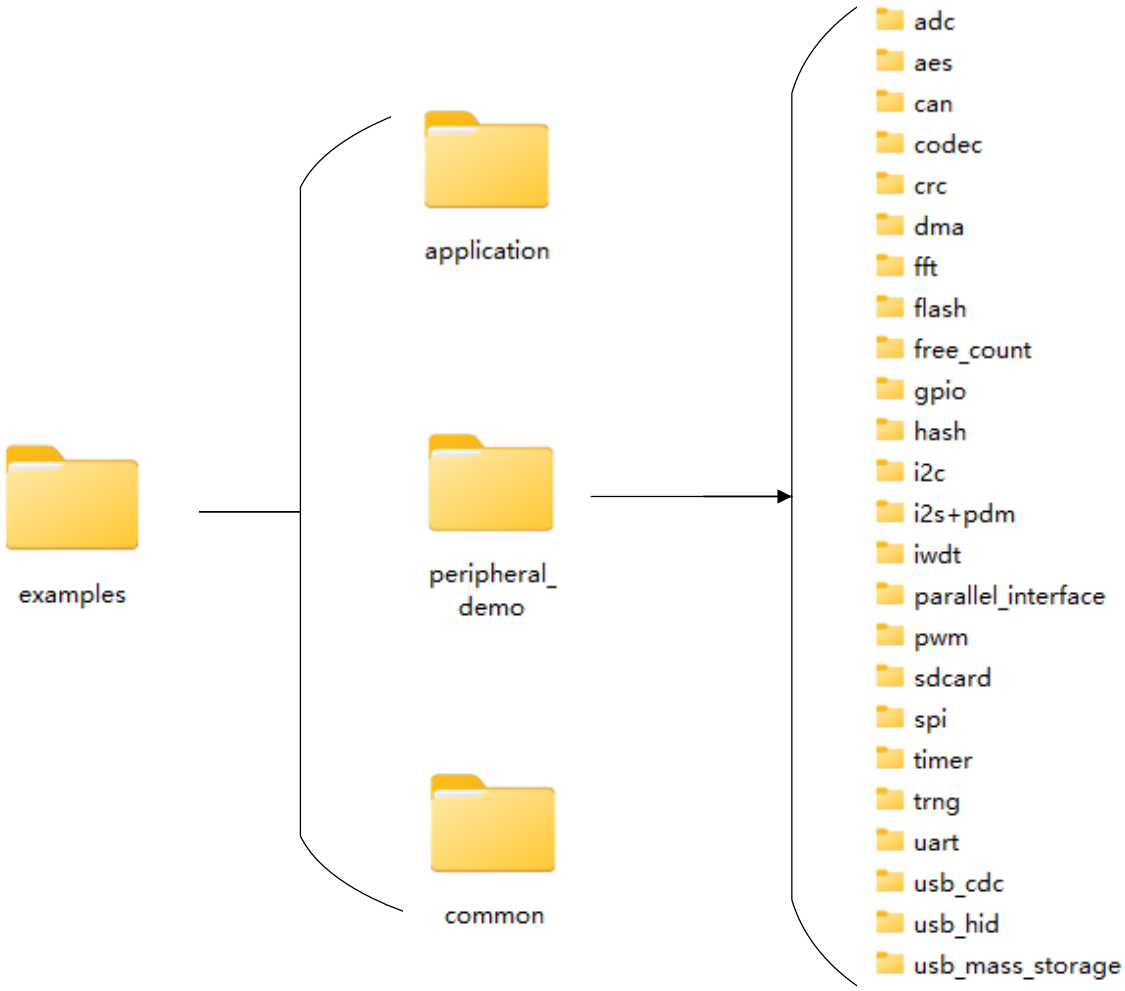
2.2. components 文件结构

components 的文件结构如下图所示。btdm 包含蓝牙 host 库等文件，modules 文件包含公共组件、第三方软件（CoreMark、FreeRTOS、LVGL.....），drivers 驱动文件包含板级驱动、不同平台编译文件、内核文件、System 驱动、启动文件、IC 驱动（SPI、UART、GPIO.....），tools 作为完整的工具包含 Flash 烧录插件、JLink 烧录设备信息配置文件、离散加载文件、Flash 固件信息字段批处理文件 post_process.bat 等。



2.3. examples 文件结构

examples 的文件结构如下图所示。examples 中包含了完整的 application 例程代码（sleep、audio、ble_simple_central、ble_simple_periphreal.....），peripheral_demo 是基于 IC 驱动编写的外设例程，以上这些都是以源码的形式提供的。

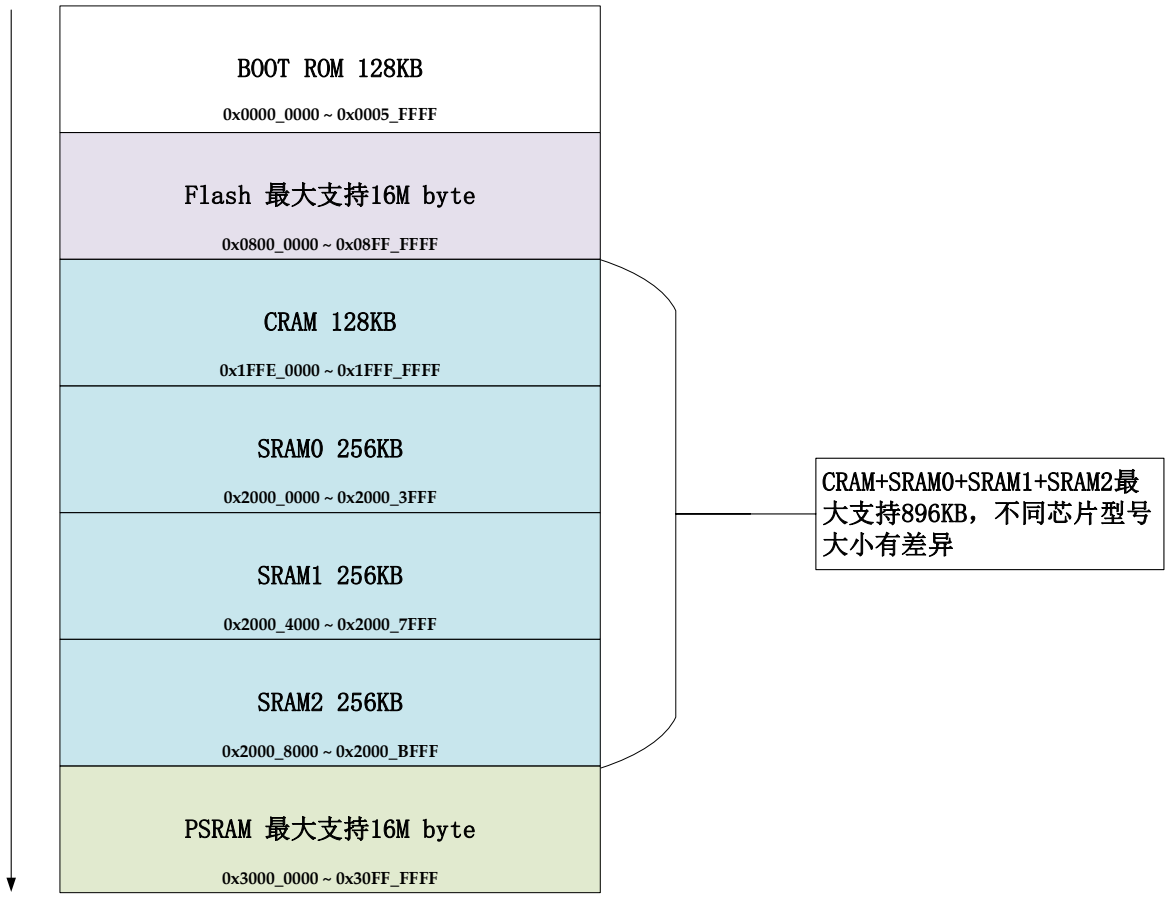


3. 存储结构

3.1. 地址空间概述

FR30xx 的存储空间主要有 ROM、FLASH、RAM 构成，其中 ROM 属于内置不可修改程序，主要实现 bootloader、支持串口烧录、安全启动、用户程序引导等功能 FLASH 通过 cache 挂接到系统 AHB 总线上，支持 XIP（片上运行），用于存储用户程序、不易失变量等；RAM 用于存储程序运行中的临时变量，对响应速度有要求的关键代码（这部分代码在系统启动时从 FLASH 拷贝过来）等。

FR30xx 系列芯片配备 128KB 的 BOOT ROM 空间，不同型号芯片 Flash、RAM 大小有所差异，PSRAM 需要通过 OSPI 扩展 RAM，增加扩展后才可通过 0x3000_0000 ~ 0x30FF_FFFF 地址直接访问，所对应的地址空间如下图所示：



3.2. FLASH 空间概述

FR30xx 内部叠封一颗 SPI Nor Flash，在内置 32KB cache 的支持下可供 CM33 XIP 运行程序。Flash 大小在不同型号的芯片上可能会有所差异。

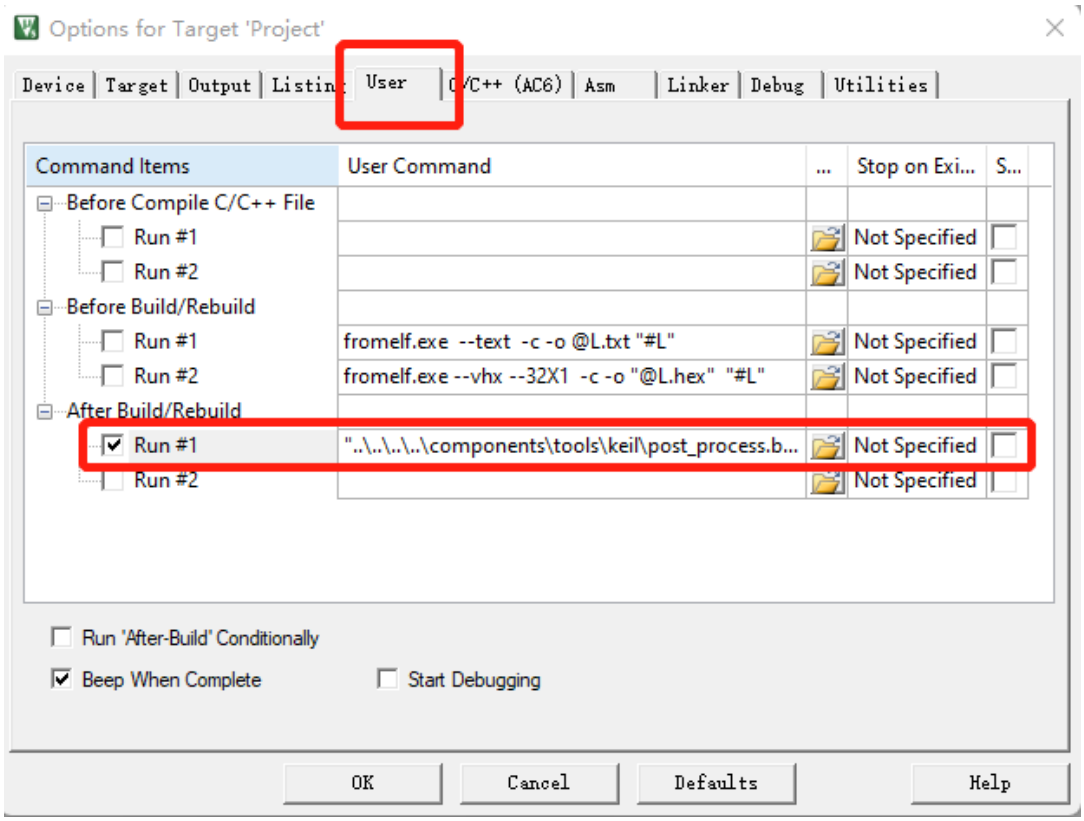
Flash 中的前 8KB 空间为预留固件信息字段，其中包含的信息用于 bootloader 引导和 OTA 升级使用。低 4KB（0x0800_0000~0x0800_0FFF，后称之为 A 区域）存储的为当前可以执行固件的信息，高 4KB（0x0800_1000~0x0800_1FFF，后称之为 B 区域）存储的为待升级固件的信息：该信息在 OTA 完成后，由 OTA 程序填写；bootloader 执行完升级之后，该区域会被 bootloader 擦除。信息字段中包含的内容如下：

偏移	名称	A 区域	B 区域	描述
0x00	image_version			固件版本号
0x04	image_offset	ignore		固件在 Flash 中的存储偏移，用于 bootloader 执行 OTA 时获取固件的存储地址
0x08	image_length			固件长度，用于计算固件的 CRC
0x0C	execute_offset			固件的执行地址，用于 bootloader 执行用户程序引导时获取固件的运行地址
0x10	copy_unit	ignore		拷贝单位，可取值有 4K，8K，16K，32K，64K，128K。用于 bootloader 执行 OTA 时单次拷贝的块大小。
0x14	copy_flag_store_step	ignore		拷贝历史记录存储步进，主要用于 OTA 过程的断点处理，可取值有 4，8，16，32，64。bootloader 每拷贝一个 copy_unit，就会记录拷贝信息到记录区域，每个拷贝信息为 4 字节，该值表示存储该信息的步进。
0x18	image_crc			固件的 crc32 校验值。
0x1C	image_valid			固件有效标识
0x20	image_tlv_length			固件 hash 值和签名的长度
0x24	options			BIT0：OTA 升级时是否需要解除写保护 BIT1：是否采用易失方式操作 flash 寄存器 BIT2-6：解保护 bit 位置 BIT7：是否处理 flash 寄存器中的 CMP 位 BIT8-10：CMP 位所处位置 BIT11：写入 flash 寄存器 2 的方式 BIT12：执行 OTA 时是否开启 cache

Copyright© 2023 上海富芮坤微电子有限公司

				BIT13-16：引导用户程序前 QSPI 切换分频比 BIT17-18：引导用户程序前 QSPI 切换写类型 BIT19-21：引导用户程序前 QSPI 切换读类型 BIT22：待升级固件存储区域与执行区域是否有重叠 BIT23：引导用户程序前是否开启 cache BIT24：CMP 标志位置位还是清除 BIT25：OTA 完成之后是否重新开启写保护
0x28	info_crc			信息段 crc32 校验值
0x2C	ota_magic	ignore		待升级固件有效标识
0x30~0xFC	Reserved	ignore	ignore	Reserved
0x100~0x1000	copy_history	ignore		拷贝信息记录区域

直接采用代码工程编译出来的文件时不包含固件信息字段的，用户需要找到 components 文件结构中提到的 tools 文件，将其中的批处理文件 post_process.bat 添加到 Keil 的 User 配置中来，如下图所示在 keil 如下配置中添加"..\..\..\..\components\tools\keil\post_process.bat" "@L" "#L" "$J"。



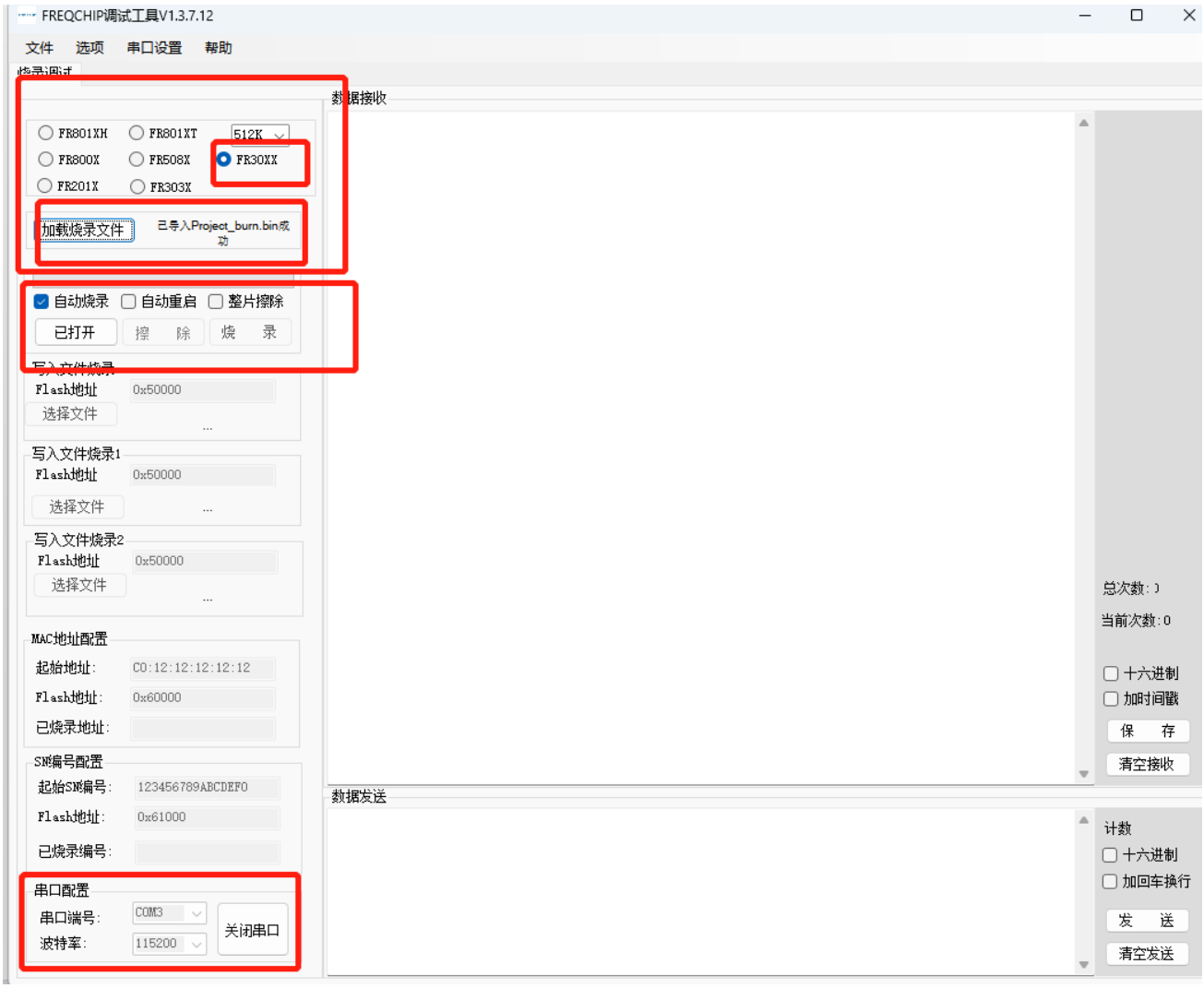
该脚本输出两个文件：Project.bin 和 Project_burn.bin，前者为原始 bin 文件，后者为添加固件信息后的固件，可以用于串口烧录。注意：该脚本采用默认值对前面描述的固件信息内容进行填写。

4. 烧录

4.1. 串口烧录

FR30xx 内置的 ROM code 具有基础的 bootloader 功能，支持串口烧录、安全启动、用户程序引导等功能。芯片上电之后首先执行 ROM code，通过 PB4（RXD）和 PB5（TXD）尝试与上位机握手，握手成功之后就可以进行程序的烧录等功能。

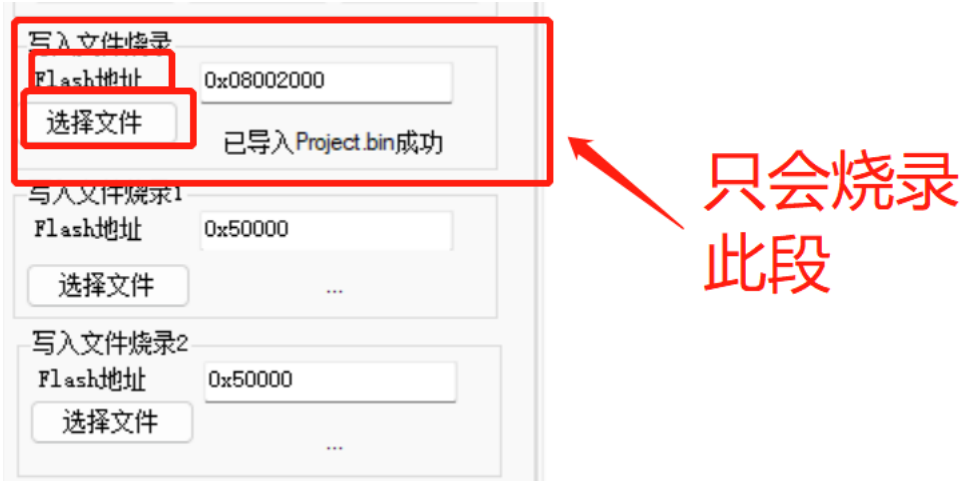
1. 加载烧录文件烧录：适用于 3.2 中提到的 Project_burn.bin 烧录。打开烧录工具点击加载烧录文件导入 Project_burn.bin 文件，最后烧录工具按照如下图所示配置，按下开发板的复位键即可进行烧录。



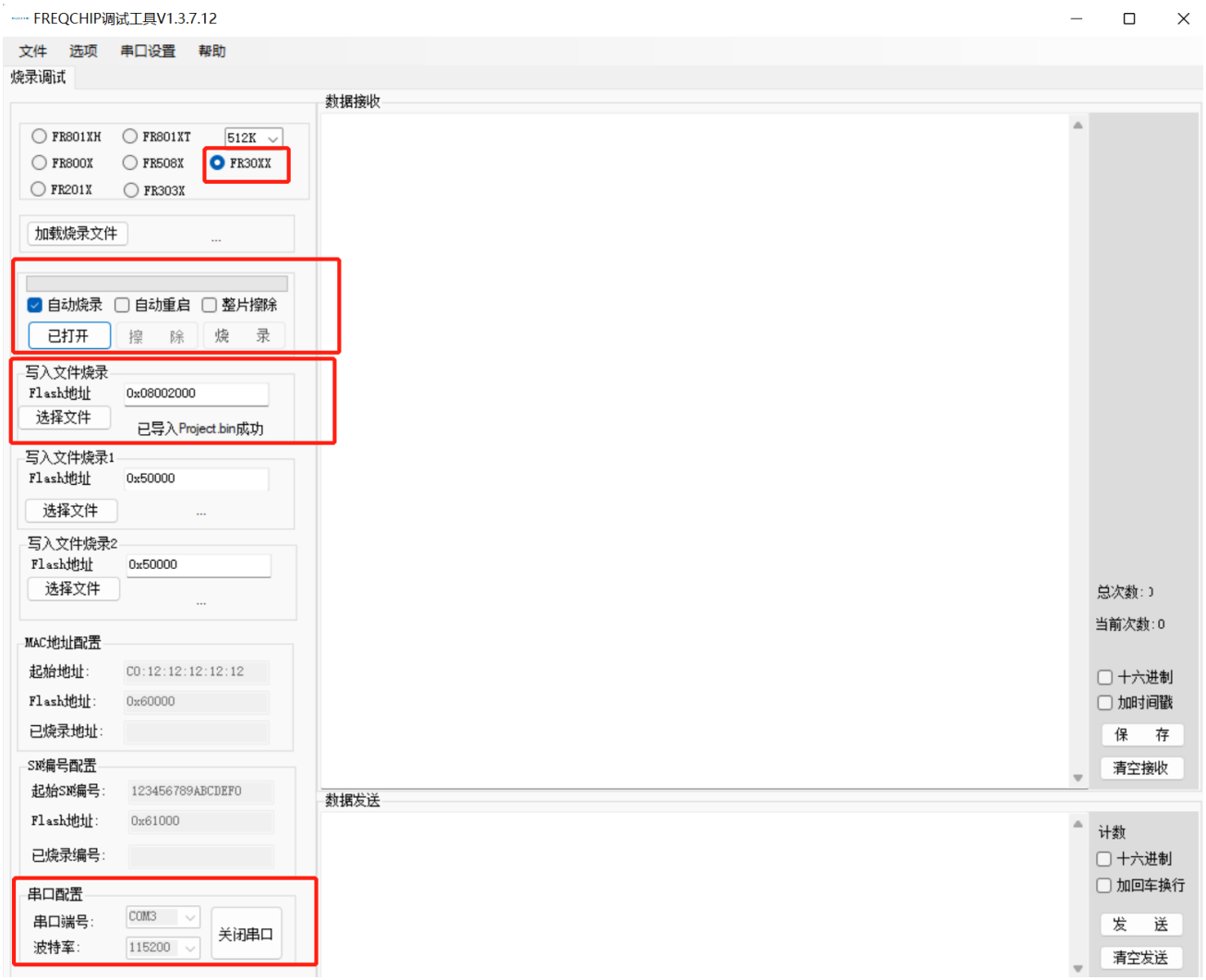
2. 指定文件烧录：适用于 3.2 中提到的 Project.bin 烧录。打开烧录工具在选项中选择烧录指定文件。



选择完成后烧录工具会出现三段写入文件烧录，他们是按顺序在指定 Flash 地址进行烧录的，此功能用户可以自主选择。想要烧录几段就写入 Flash 地址和选择文件即可，不点击选择文件导入待烧录文件则不烧录。



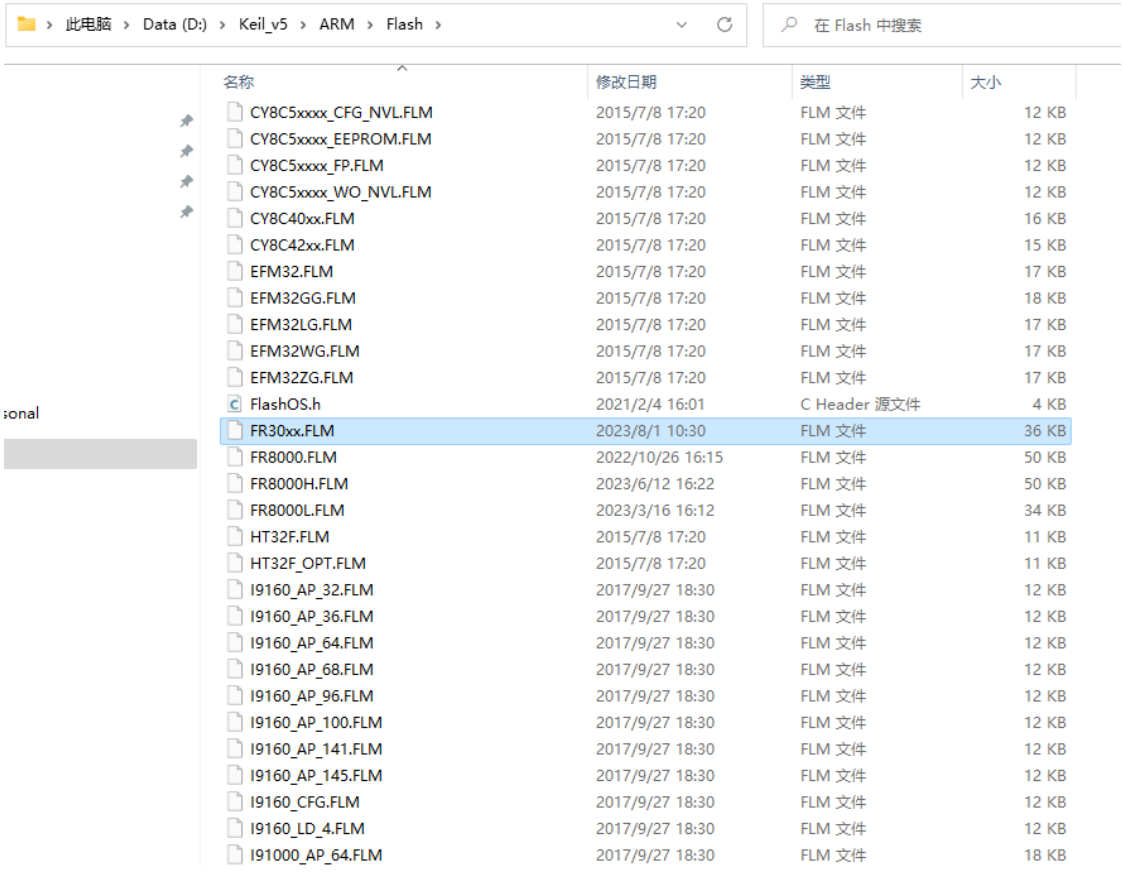
最后烧录工具按照如下图所示配置，按下开发板的复位键即可进行烧录。



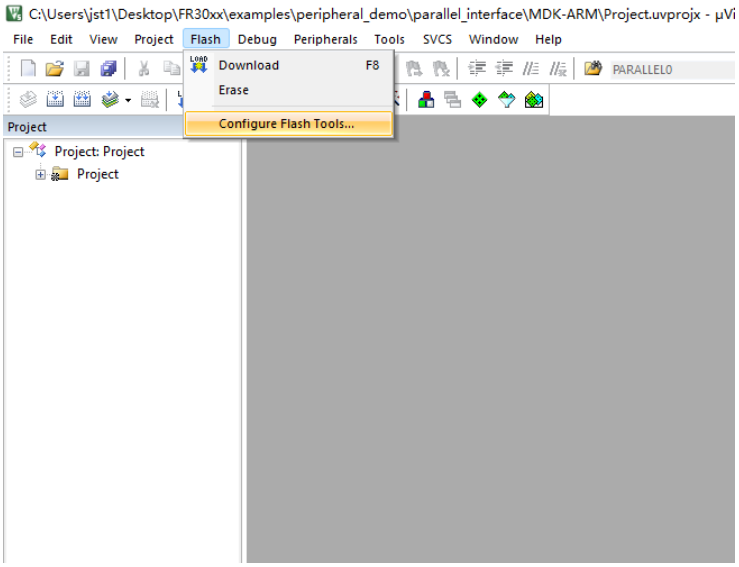
4.2. 基于 Keil 烧录

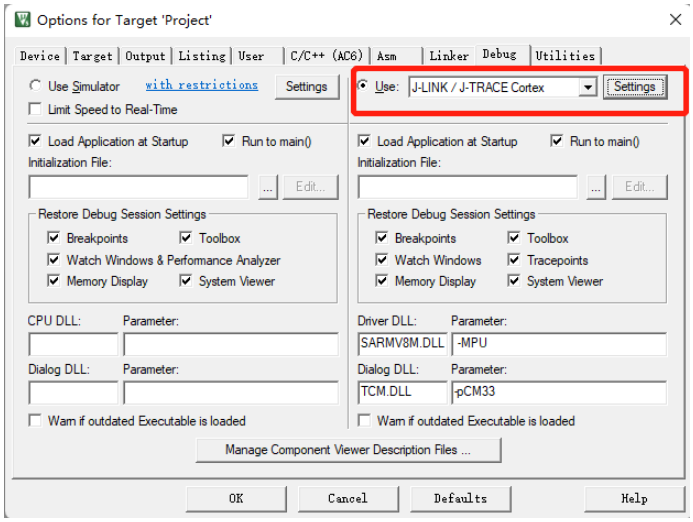
除了串口烧录芯片还可以使用 J-Link 烧录，芯片的 PB10（SWCLK0）和 PB11（SWD）为调试接口，在芯片工作在正常模式下，且 IOMUX 未修改的情况下可以通过该调试接口进行调试和程序下载。

1. 用户需要找到 components 文件结构中提到的 tools 文件，将其中的 FR30xx.FLM 添加到 Keil 安装目录下的 Flash 插件文件夹下，如下图所示。

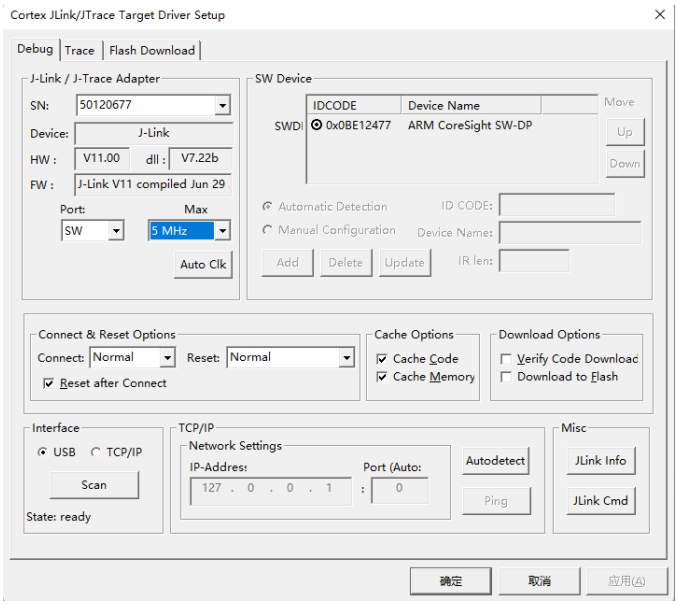


2. 选用 J-Link 作为调试工具，Keil 工具菜单栏 Flash，下拉选择 Config Flash Tools 在 Debug 子选项卡，选择 J-LINK/J-TRACE Cortex 选项。

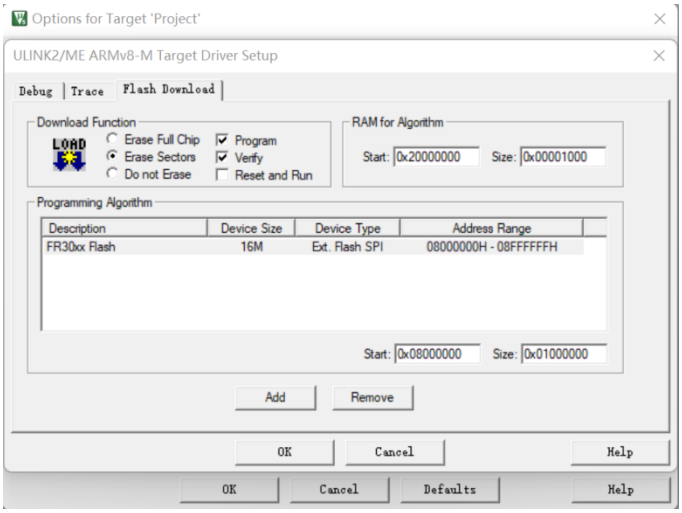

C:\Users\jst1\Desktop\FR30xx\examples\peripheral_demo\parallel_interface\MDK-ARM\Project.uvprojx - µVi
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Download F8
Erase
Configure Flash Tools...
Project
Project: Project
Project


Options for Target 'Project'
Device Target Output Listing User C/C++ (AC6) Asm Linker Debug Utilities
Use Simulator with restrictions Settings
Limit Speed to Real-Time
Use: J-LINK / J-TRACE Cortex Settings
Load Application at Startup Run to main()
Initialization File:
Edit...
Restore Debug Session Settings
Breakpoints Toolbox
Watch Windows & Perfomance Analyzer
Watch Windows Tracepoints
Memory Display System Viewer
CPU DLL: Parameter:
Driver DLL: Parameter:
SARMV8M.DLL -MPU
Dialog DLL: Parameter:
TCM.DLL -pCM33
Warn if outdated Executable is loaded
Manage Component Viewer Description Files ...
OK Cancel Defaults Help

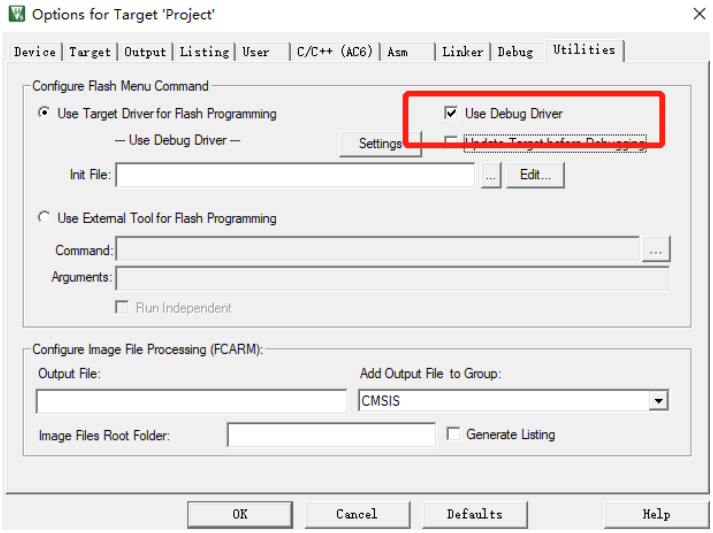
3. 配置调试方式为 SW。


Cortex JLink/JTrace Target Driver Setup
Debug Trace Flash Download
J-Link / J-Trace Adapter
SN: 50120677
Device: J-Link
HW : V11.00 dll : V7.22b
FW : J-Link V11 compiled Jun 29
Port: Max
SW 5 MHz
Auto Clk
SW Device
IDCODE Device Name
SWDI 0x0BE12477 ARM CoreSight SW-DP
Move Up Down
Automatic Detection ID CODE:
Manual Configuration Device Name:
Add Delete Update IR len:
Connect & Reset Options
Connect: Normal Reset: Normal
Reset after Connect
Cache Options
Cache Code
Cache Memory
Download Options
Verify Code Download
Download to Flash
Interface
USB TCP/IP
Scan
State: ready
TCP/IP
Network Settings
IP-Address: Port (Auto:
127 . 0 . 0 . 1 : 0
Autodetect
Ping
Misc
JLink Info
JLink Cmd
确定 取消 应用(A)

4. 在 keil 的 flash download 中选择 FR30xx.FLM。



5. 配置使用 Debug Driver 进行 flash 的烧录。

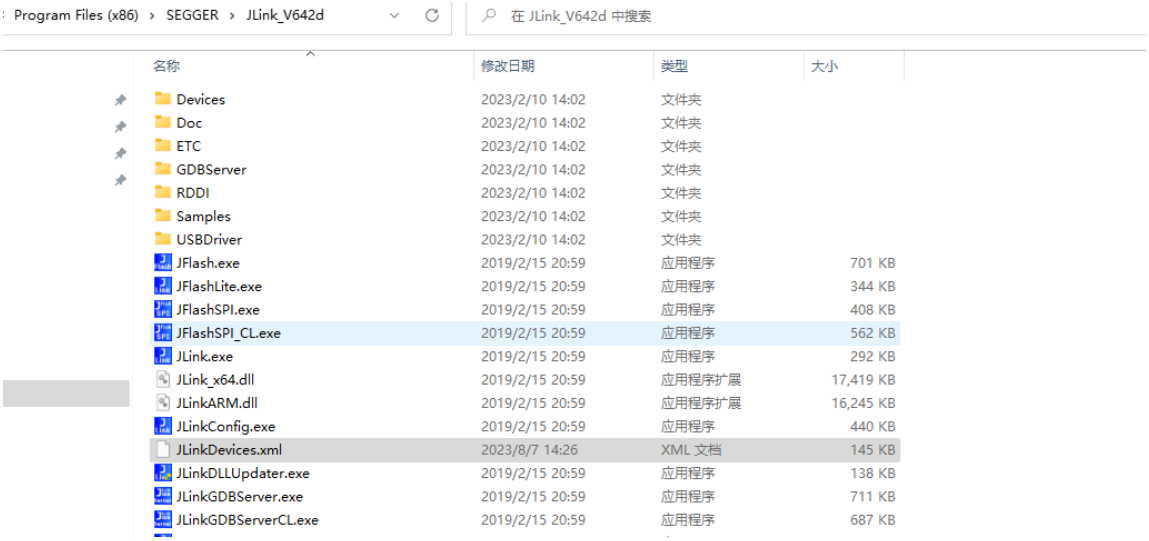


芯片 Flash 的烧录起始地址为 0x0800_2000，需要预留 0x2000 的固件信息字段。必须注意 MCU_CLK 工作频率超过 24MHZ 时无法进行调试，该调试接口可以通过更改内部 efuse 配置实现永久关闭。烧录时需要注意系统不能处于睡眠状态，且 SWCLK 和 SWO 功能引脚未用作其他用途。

4.3. 基于 J-Flash 烧录

在使用 J-Link 烧录的基础上还提供了使用 J-Flash 工具烧录的方式。

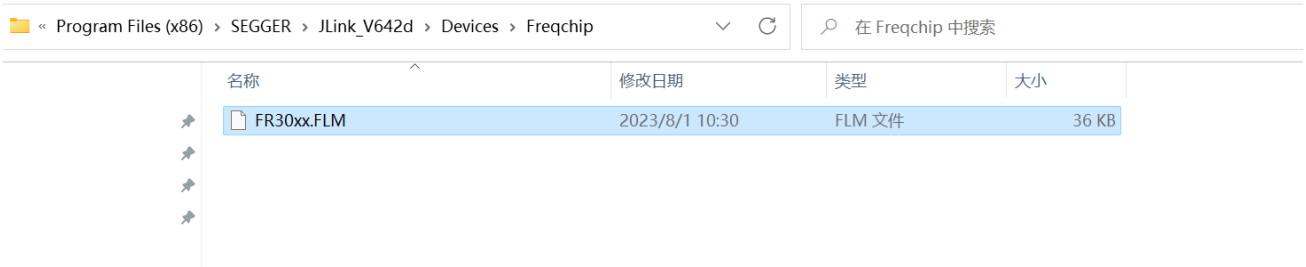
1. 找到 components 文件结构中提到的 tools 文件中的 JLinkDevices.xml。打开文件将里面的内容拷贝添加到 J-Link 安装目录下的 JLinkDevices.xml 中，如下图所示。



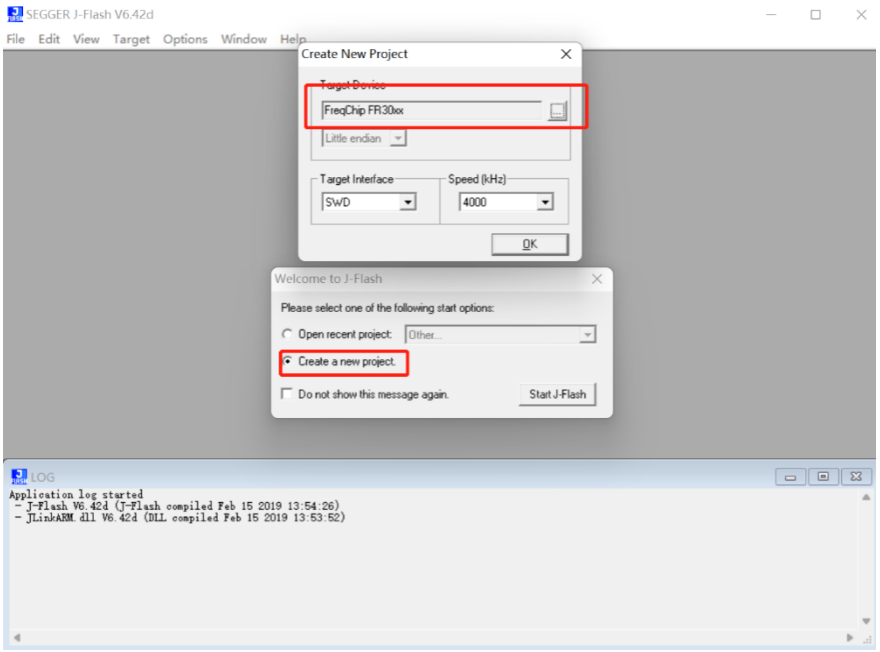
需要拷贝的芯片声明如下：

```
<DataBase>
  <!--          -->
  <!-- FreqChip -->
  <!--          -->
  <Device>
    <ChipInfo Vendor="FreqChip" Name="FR30xx" Core="JLINK_CORE_CORTEX_M33" WorkRAMAddr="0x20000000" WorkRAMSize="0x8000"/>
      <FlashBankInfo Name="Internal Flash" BaseAddr="0x08000000" MaxSize="0x1000000" Loader="Devices/Freqchip/FR30xx.FLM" LoaderType="FLASH_ALGO_TYPE_OPEN" AlwaysPresent="1"/>
    <FlashBankInfo Name="External Flash" BaseAddr="0x10000000" MaxSize="0x8000000" Loader="Devices/Freqchip/FR30xx_ext.FLM" LoaderType="FLASH_ALGO_TYPE_OPEN" AlwaysPresent="1"/>
  </Device>
</DataBase>
```

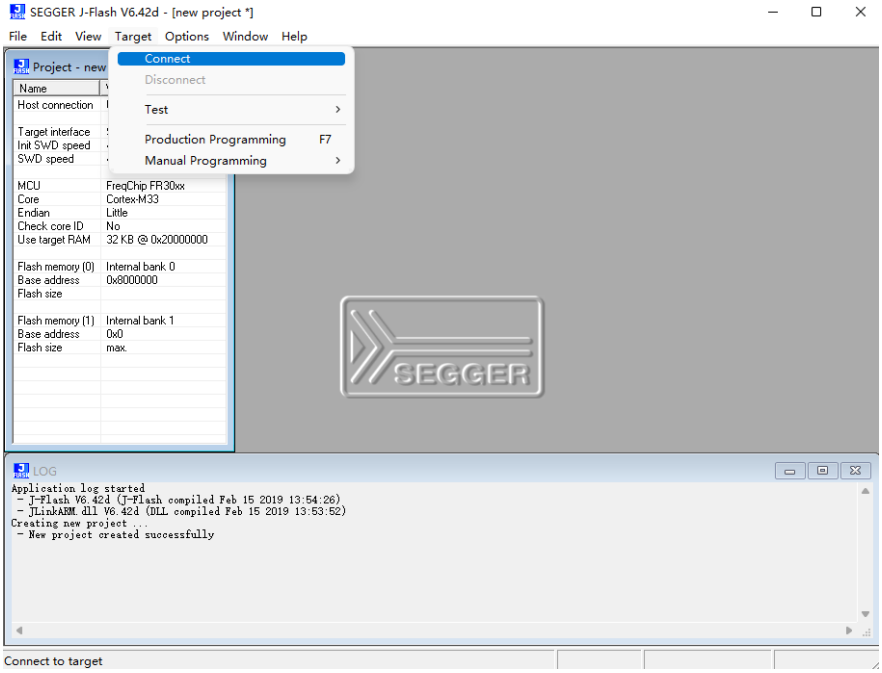
2. 在 JLink 安装目录中创建 SEGGER\JLink\Devices\Freqchip 文件夹，并将 FR30xx.FLM 文件拷贝到该文件夹下：



3. 打开 J-Flash，创建新工程，Target Device 选择 FreqChip FR30xx。



4. 进入如下界面，菜单栏选择 Target->connect 连接待烧录芯片。



5. 菜单栏选择 File->Open data file，并输入起始烧录地址，打开待烧录文件

6. 菜单栏选择 Target->Production Programming 或者 Target->Maual Programming->Program/Program & Verify 执行文件的烧录。

5. System API

5.1. System API 概述

System API 相关可以在 SDK 文件中的 components \drivers\device\fr30xx\ system_fr30xx.h 中找到。其中包括系统初始化、时钟配置、系统及外设时钟源获取、DMA 请求编号选择、微秒级别定时器、Flash Cache 的使能与关闭配置、系统 Sleep 相关。其中，系统时钟相关可以参考 FR30xx 系列时钟配置应用笔记。系统初始化 API 已在 startup_fr30xx.s 中调用，用户无需关心。

```
/* SystemInit */
void SystemInit(void);

/* System_CORE_HSCLK_config */
/* System_SPLL_config */
/* System_AUPLL_config */
/* System_MCU_clock_Config */
void System_CORE_HSCLK_config(System_CORE_HSCLKConfig_t *COREHConfig);
int System_SPLL_config(System_PLLConfig_t *PLLConfig, uint32_t fu32_timeout);
int System_AUPLL_config(System_PLLConfig_t *PLLConfig, uint32_t fu32_timeout);
void System_MCU_clock_Config(System_ClkConfig_t *ClkConfig);

/* Get system Core/DSP/SPLLCLK/AUPLLCLK/CORE_HSCLK clock. unit HZ */
uint32_t system_get_CoreClock(void);
uint32_t system_get_DSPClock(void);
uint32_t system_get_CORE_HSCLK(void);
uint32_t system_get_SPLLCLK(void);
uint32_t system_get_AUPLLCLK(void);
uint32_t system_get_LPRCCLK(void);
void system_set_LPRCCLK(uint32_t);
/* System_get_peripheral_clock */
uint32_t system_get_peripheral_clock(per_clock_index_t peripheral);

/* system_dmac_request_id_config */
void system_dmac_request_id_config(dmac_request_source_t fe_source, dmac_request_id_t fe_id);

/* system_delay_us */
void system_delay_us(uint32_t fu32_delay);

/* Enable and Disable cache */
void system_cache_enable(bool invalid_ram);
void system_cache_disable(void);

/* system_prevent_sleep_set*/
/* system_prevent_sleep_clear*/
/* system_prevent_sleep_get*/
void system_prevent_sleep_set(uint32_t type);
void system_prevent_sleep_clear(uint32_t type);
uint32_t system_prevent_sleep_get(void);
```

5.2. 外设时钟源获取

SDK 配备了完整的外设时钟获取接口，在完成系统时钟配置后如果需要获取某个外设的时钟可以调用 uint32_t system_get_peripheral_clock(per_clock_index_t peripheral)。per_clock_index_t 在调用时需要在下图中选择。值得注意的是当系统时钟改变时需要重新调用进行获取。

```
/*  peripheral clock select*/
typedef enum
{
    PER_CLK_UARTx,          // Uart0/Uart1/Uart2/Uart3/Uart4/Uart5
    PER_CLK_GPIOx,          // GPIOA/GPIOB/GPIOC/GPIOD
    PER_CLK_I2Cx,           // I2C0/I2C1/I2C2/I2C3/I2C4/I2C5
    PER_CLK_I2Sx,
    PER_CLK_TIMER01,        // Timer0/Timer1
    PER_CLK_TIMER23,        // Timer2/Timer3
    PER_CLK_SPIMx,
    PER_CLK_SPIS,
    PER_CLK_SPIMX8_0,
    PER_CLK_SPIMX8_1,
    PER_CLK_PWM,
    PER_CLK_SBC,
    PER_CLK_PDM0,
    PER_CLK_PDM1,
    PER_CLK_PDM2,
    PER_CLK_PARALLEL,
    PER_CLK_QSPI0,
    PER_CLK_QSPI1,
    PER_CLK_OSPI,
    PER_CLK_SDIOH0,
    PER_CLK_SDIOH1,
    PER_CLK_CANx,           // CAN0/CAN1
    PER_CLK_SPDIF,
}per_clock_index_t;
```

5.3. 微秒级别定时器

外设部分的 Timer 使用时需要初始化，因此 SDK 中封装了一个十分方便的 32 位微秒级别定时器，无需关心时钟和初始化。使用时调用 void system_delay_us(uint32_t fu32_delay)，具体也可以参考 SDK 文件 examples\peripheral_demo\free_count 中的使用例程。

5.4. DMA 请求编号配置

DMA 的使用场景，大多数为内存到外设；外设到内存这一类。当目标或地址为外设时，需要用户手动分配 DMA 的请求编号。SDK 中已经做好了封装，每个外设 IP 都可以选择 1~1

5 范围内的请求。调用 system_dmac_request_id_config(dmac_request_source_t fe_source, dmac_request_id_t fe_id) 时需要配置 dmac_request_id_t 和 dmac_request_source_t，具体也可以参考 SDK 文件 examples\peripheral_demo\dma 中的使用例程。

下面是 dmac_request_id_t 参数选择，必须注意 DMA0 和 DMA1 的请求编号是分开配置的。

```
/* DMA0/1 request ID */
typedef enum
{
    /* following ids are used for DMA0 */
    DMA0_REQUEST_ID_1 = 1,
    DMA0_REQUEST_ID_2,
    DMA0_REQUEST_ID_3,
    DMA0_REQUEST_ID_4,
    DMA0_REQUEST_ID_5,
    DMA0_REQUEST_ID_6,
    DMA0_REQUEST_ID_7,
    DMA0_REQUEST_ID_8,
    DMA0_REQUEST_ID_9,
    DMA0_REQUEST_ID_10,
    DMA0_REQUEST_ID_11,
    DMA0_REQUEST_ID_12,
    DMA0_REQUEST_ID_13,
    DMA0_REQUEST_ID_14,
    DMA0_REQUEST_ID_15,
    /* following ids are used for DMA1 */
    DMA1_REQUEST_ID_1 = 17,
    DMA1_REQUEST_ID_2,
    DMA1_REQUEST_ID_3,
    DMA1_REQUEST_ID_4,
    DMA1_REQUEST_ID_5,
    DMA1_REQUEST_ID_6,
    DMA1_REQUEST_ID_7,
    DMA1_REQUEST_ID_8,
    DMA1_REQUEST_ID_9,
    DMA1_REQUEST_ID_10,
    DMA1_REQUEST_ID_11,
    DMA1_REQUEST_ID_12,
    DMA1_REQUEST_ID_13,
    DMA1_REQUEST_ID_14,
    DMA1_REQUEST_ID_15,
}dmac_request_id_t;
```

下面是 dmac_request_source_t 参数选择。

```
/* DMA0/1 request source */
typedef enum
{
    SPIMX8_0_RX,        /* 0 */
    SPIMX8_0_TX,        /* 1 */
    SPIMX8_1_RX,        /* 2 */
    SPIMX8_1_TX,        /* 3 */

    SPIM0_RX,           /* 4 */
    SPIM0_TX,           /* 5 */
    SPIM1_RX,           /* 6 */
```

```
    SPIM1_TX,           /* 7 */

    SPIM2_RX,           /* 8 */
    SPIM2_TX,           /* 9 */
    SPISRX,             /* 10 */
    SPISTX,             /* 11 */

    SPISX4_RX,          /* 12 */
    SPISX4_TX,          /* 13 */
    UART0_RX,           /* 14 */
    UART0_TX,           /* 15 */

    UART1_RX,           /* 16 */
    UART1_TX,           /* 17 */
    UART2_RX,           /* 18 */
    UART2_TX,           /* 19 */

    UART3_RX,           /* 20 */
    UART3_TX,           /* 21 */
    UART4_RX,           /* 22 */
    UART4_TX,           /* 23 */

    UART5_RX,           /* 24 */
    UART5_TX,           /* 25 */
    BLEND_RGB0,         /* 26 */
    BLEND_RGB1,         /* 27 */

    BLEND_MASK,         /* 28 */
    BLEND_ORGB,         /* 29 */
    SPDIF_TRANS,        /* 30 */
    PARALLEL_INTERFACE, /* 31 */

    I2S0_RX_LEFT,       /* 32 */
    I2S0_TX_LEFT,       /* 33 */
    I2S0_RX_RIGTH,      /* 34 */
    I2S0_TX_RIGTH,      /* 35 */

    I2S1_RX_LEFT,       /* 36 */
    I2S1_TX_LEFT,       /* 37 */
    I2S1_RX_RIGTH,      /* 38 */
    I2S1_TX_RIGTH,      /* 39 */

    I2S2_RX_LEFT,       /* 40 */
    I2S2_TX_LEFT,       /* 41 */
    I2S2_RX_RIGTH,      /* 42 */
    I2S2_TX_RIGTH,      /* 43 */

    PDM0_RX,            /* 44 */
    PDM1_RX,            /* 45 */
    PDM2_RX,            /* 46 */
    CODEC_RX_LEFT,      /* 47 */

    CODEC_TX_LEFT,      /* 48 */
    CODEC_RX_RIGTH,     /* 49 */
    CODEC_TX_RIGTH,     /* 50 */
    CODEC_RX_0,         /* 51 */

    CODEC_RX_1,         /* 52 */
    SBC_DEC_IN,         /* 53 */
```

```
    SBC_DEC_OUT,          /* 54 */
    SBC_ENC_IN,           /* 55 */

    SBC_ENC_OUT,          /* 56 */
    YUV2RGB_IN,           /* 57 */
    YUV2RGB_OUT,          /* 58 */
}dmac_request_source_t;
```

5.5. 系统睡眠

SDK 中提供 system_prevent_sleep_set 和 system_prevent_sleep_clear 来开启和关闭睡眠功能、system_prevent_sleep_get 来获取睡眠状态。需要注意 system_prevent_sleep_set 调用之后并不会立即进入睡眠状态，而是使能是否可以睡眠的监测功能。

6. 变更历史

版本号	日期	更新内容
V1.0	2023.8.7	首版
V1.1	2023.11.28	修改 4.1 章节

7. 联系信息

公司：上海富芮坤微电子有限公司

地址：中国(上海)自由贸易试验区碧波路 912 弄 8 号 501-A 室

电话：+86-21-5027-0080

Website: www.freqchip.com

Sales Email: sales@freqchip.com

本文档的所有部分，其著作产权归上海富芮坤微电子有限公司（简称富芮坤）所有，未经富芮坤授权许可，任何个人及组织不得复制、转载、仿制本文档的全部或部分。富芮坤保留在不另行通知的情况下随时对产品或本文档进行更改、修正、增强的权利。购买者应在订购前获得富芮坤产品的最新相关资料。